# 11-768 HW2: Evaluating a Data-Visualization Agent

zhuermu (self-study, not an enrolled submission)

**Environment deviations.** Modal was unavailable, so the work ran on AWS. The validator model is unchanged: `Qwen/Qwen3-VL-30B-A3B-Instruct-FP8` served by vLLM 0.13.0 on one L40S (g6e.xlarge) with the exact arguments in `infrastructure/validator_model.py`. The three Part 3 agents were substituted by Amazon Bedrock models: `mistral.ministral-3-3b-instruct` for Qwen2.5-Coder-3B (no Bedrock equivalent; the smallest comparable model), `mistral.ministral-3-14b-instruct` and `zai.glm-4.7-flash` (the same models as the handout's, hosted). The mini-SWE-agent 2.4.5 scaffold, prompt, step limit (20), command timeout (120 s) and wall-time limit were copied from `agent_runner.py` and run in a local Docker container with the same package pins. Because the recorded agent models differ from the fixed ones, `check-submission` rejects the runs on that one check by design.

## 1 Part 1: Examine Seed Agent Runs

### 1.1 Comparison of Ground-Truth and Validator Labels

| Family | MCC | TP | TN | FP | FN |
|---|---|---|---|---|---|
| execution_failure | −0.002 | 4 | 65 | 33 | 8 |
| wrong_data | 0.000 | 0 | 73 | 0 | 25 |
| wrong_chart | 0.110 | 3 | 27 | 0 | 68 |
| hard_to_read | 0.000 | 0 | 59 | 0 | 39 |
| macro-MCC | 0.027 | | | | |

The baseline predicted "acceptable" for 68 runs (only 3 truly are), `execution_failure` for 37 and `wrong_chart` for 5. It never predicted `wrong_data` or `hard_to_read`. Disagreements concentrate in two places: spurious `execution_failure` flags on runs that did produce a figure, and missed design/data violations on everything else.

### 1.2 Validator Error Type 1: inverted execution answers that hide everything else

All 33 `execution_failure` false positives have a `figure.png`; in 32 of them the model's own explanation says the agent succeeded, e.g. run 022 ("The agent successfully completed the task…two side-by-side subplots") and run 062 ("…generated a vertical violin plot with all the required elements"), both labelled acceptable. The question is a long negated disjunction ("Did the agent crash, exhaust its turns, or give up without producing a valid figure?"); the model answers "YES" meaning "yes, it produced one", and `_ask` reads only the first token. Because `validate()` returns early on this flag, 25 real labels in 20 of these runs were never examined (14 `wrong_chart`, 8 `hard_to_read`, 3 `wrong_data`), e.g. run 004 (third panel's x axis linear instead of log; fourth panel

missing the cubing step) and run 135 (category labels drawn over their branches). Conversely the 8 false negatives (010, 053, 065, 089, 096, 168, 181, 208) have no figure at all.

### 1.3 Validator Error Type 2: explicit, checkable requirements waved through

65 runs predicted acceptable carry real errors (54 `wrong_chart`, 28 `hard_to_read`, 22 `wrong_data`). Many are literal parameters in the instructions: 30° x-tick rotation drawn horizontal (009, 038), 120 dpi saved at 100 (060, 130), a forbidden `jet` colormap (051, 106, 130), a requested legend absent (016, 028), a y axis that must start at 0 (112, 126), horizontal bars drawn vertical (142). Each family gets one vague question plus the raw trajectory string; a weak VLM facing a long context and "does it fail any part" defaults to "no". Most of these properties are hard or impossible to read from pixels but are exact values in the figure object.

### 1.4 Proposed Validator Changes

(1) Decide execution deterministically from the artifact (missing, undecodable, tiny or single-colour PNG), and never short-circuit the other families when a figure exists. (2) Re-execute the agent's commands under instrumentation, record the Matplotlib artist tree, check literal requirements in code, and ask the model to compare the request against those measurements one requirement at a time, with structured JSON output instead of a YES/NO prefix.

## 2 Part 2: Improve Your Validator

### 2.1 Implemented Changes

**Change 1 (deterministic).** `judge_execution` checks the PNG only. On the seed set "no figure $\Leftrightarrow$ execution_failure" holds for all 110 runs; a blank-canvas and decode check guard the private set. When it fires the validator returns that family alone (it is terminal in the ground truth); otherwise all three remaining families are always judged.

**Change 2 (deterministic + model).** Every bash command in the trajectory is replayed, in order and each in a fresh shell like the scaffold, inside a Docker sandbox (`--network none`, 2 GB, non-root) built with the agent image's pins. A `sitecustomize.py` on `PYTHONPATH` patches `Figure.savefig` and dumps the handout's `figure_manifest` plus fields it lacks: the DPI actually passed to `savefig`, tick rotation, spine visibility, and the pixel box, colour and background of every text. 97 of 98 figure-producing seed runs replay, all at the original image size (run 191 needs `pip install`, which the offline sandbox refuses). Rules then check DPI, figure size, forbidden/requested colormaps, tick rotation, legend/grid/colour bar presence, y-from-zero, bar orientation, 3D and polar projections, and hidden spines. Finally the model receives the request, the input files and a compact JSON of the measurements (no image) and labels each explicit requirement met/violated/unknown; only "violated" items with a valid family are kept. Readability was left as the baseline's so the effect of the two changes is attributable.

### 2.2 Seed-Set Performance vs. Baseline

| Family | Baseline | Part 2 | Part 4 (final) |
|---|---|---|---|
| execution_failure | −0.002 | 1.000 | 1.000 |
| wrong_data | 0.000 | 0.157 | 0.041 |
| wrong_chart | 0.110 | 0.414 | 0.479 |
| hard_to_read | 0.000 | −0.242 | 0.481 |
| macro-MCC | 0.027 | 0.332 | 0.500 |

### 2.3 Qualitative Analysis of Targeted Error Types

Type 1 is fully resolved (12/12 TP, 0 FP). Type 2 is partly resolved: `wrong_chart` rises to 55 TP / 9 FP; rules alone (no model) reach 26 TP / 0 FP. Run 009's rotation and 004's two errors are now caught. *Regression:* `hard_to_read` drops to −0.242 (0 TP, 8 FP). Nothing changed in that judge; removing the early return exposed the baseline's readability question on 33 more runs, so the baseline's 0.000 meant "never asked", not "never wrong". *False positive, run 082 (wrong_data):* the model claims auth's median is 97 ("average of 95 and 99") while the box shows 101.5; the true median of 82, 91, 95, 99, 104, 110, 118, 240 is $(99 + 104)/2 = 101.5$. The judge did the arithmetic wrong and blamed the agent; most of the 15 `wrong_data` FPs are of this kind. *False negative, run 016 (wrong_chart):* the label says a requested legend is missing. My legend rule's `for/else` branch was inverted, so it only checked when the request said "no legend". Fixed in Part 4.

## 3 Part 3: Design New Evaluation Tasks

### 3.1 Anticipated Generalization Weaknesses

**Derived aggregation** (`task_class = derived-aggregation`): the validator has no deterministic `wrong_data` rules and the model only sees summarised series, so errors in filtering, sorting, cumulative or rolling computations should slip through. **Dense text layout** (`dense-text-layout`): readability is still judged by the baseline VLM question, which never produced a correct positive on the seed set; many long labels or annotations should expose it.

### 3.2 Task Design

All inputs are synthetic CSVs generated with a fixed seed.

- `regional-revenue-share` (A): 60 orders; revenue = units × price, drop `Sample`, percentage share per region, horizontal bars with the largest at the top, value labels, 8×5 in at 100 dpi. Tests filter + derived column + normalisation + sort direction.
- `lyon-rolling-temperature` (A): two cities, 90 days, four missing Lyon days; filter, drop missing, 7-day trailing mean with no value for the first six rows. Tests subtle window semantics.
- `cumulative-signups-by-channel` (A): 16 weeks × 3 channels; cumulative sums stacked in a fixed order. Tests per-group cumulation and stack order.
- `department-headcount-long-labels` (B): 14 departments with long names in a 7×4 in figure, value labels on every bar. Tests crowded tick labels.

- `african-cities-annotated-scatter` (B): 25 labelled points on a log-x 6×4.5 in scatter, with clusters that collide unless placed with care.

### 3.3 Agent Runs and Labeling

Each task ran once per agent (15 runs, all produced a figure). I labelled each run from the image, the final script, and expected values recomputed from the inputs, then checked against the replayed manifest. All three agents drew the correct shares in `regional-revenue-share` but put the largest at the bottom (`barh` draws the first row at the bottom): `wrong_data` (wrong order). In `lyon` the 3B agent's mean is correct but its 86 horizontal date ticks overprint (`hard_to_read`); the 14B and GLM agents used `min_periods=1` (`wrong_data`). The 3B agent's cumulative chart assigns columns onto a misaligned index, so nothing is drawn (`wrong_data`). The 3B agent's headcount chart squeezes the bars into 18% of the canvas (`hard_to_read`); the 40–45° labels of the other two were judged readable. All three scatters have overlapping city labels (`hard_to_read`). Ambiguity: the 3B cumulative chart also inverts the y axis; with no data drawn I labelled only `wrong_data`. Labels are a first-pass review and are pending a second human check.

### 3.4 Validator Performance on New Tasks

| Family | Part 2 (pre-iteration) | Part 4 (final) |
|---|---|---|
| execution_failure | 0.000* | 0.000* |
| wrong_data | −0.068 | 0.739 |
| wrong_chart | 0.000* (13 FP) | 0.000* (3 FP) |
| hard_to_read | −0.189 | 0.853 |
| macro-MCC | −0.064 | 0.398 |

*Insufficient support: no positive labels in this family. Before iteration, 13 of 15 runs got a spurious `wrong_chart`. The causes were validator weaknesses, not agent ones: my facts JSON cut annotations at 10, so the model reported "only 10 of 25 annotations" on every scatter; it reported the colormap `viridis` of a uniformly grey scatter as "dots not grey"; and it judged sort order inconsistently. The agents' real failures (sort direction, window semantics, label collisions) were mostly missed.

### 3.5 Harbor Packaging

All five tasks score 1.0 with `-a oracle`; `-a nop` scores 0 (checked on two tasks). The handout example gives oracle 1, nop 0, and `illegible` 1, as documented. Each verifier re-runs `plot.py` with the manifest hook and compares hand-derived literals. **Mutant `regional-revenue-share/largest_at_bottom`: reward 0.** Exactly the agents' mistake: correct shares, labels, title and size, but sorted descending so the largest bar is at the bottom. `test_s2_largest_share_at_the_top` rejects it from the y-tick order. **Mutant `african-cities-annotated-scatter/label_pileup`: reward 1.0.** All values, all 25 labels anchored at their points, log scale, exact text and size are right, but a fixed 9 pt offset makes Kigali/Gaborone and Douala/Bamako/Johannesburg/Cape Town print over each other, violating "must not overlap". No assertion measures text collisions, so it passes. **Which families a deterministic verifier can decide:** `execution_failure` yes (file present and decodable); `wrong_data` yes for a known task (literal expected values via the sidecar and the re-rendered artists); `wrong_chart` mostly (type, scales, text, size, legend are in the artist tree), less so for free-form style requests; `hard_to_read` only partly: overlap and clipping of boxes can be measured, but

whether a human can read the result is a perceptual judgement, which is what the label_pileup mutant demonstrates. **Why a per-task verifier cannot grade the validator:** it encodes the answer for one task; the private set contains tasks and chart types nobody wrote a verifier for, so grading needs a task-agnostic judge.

# 4 Part 4: Iterate on Your Design

## 4.1 Additional Improvement Motivated by Part 3

(a) Readability is now measured from the replayed figure instead of asked: overlapping text (rotation-aware spacing for rotated tick labels, 20% box-overlap threshold), text clipped by the canvas, text with contrast below 1.8:1 against its local background, series drawn in the axes background colour or with zero width, and a single axes squeezed below 20% of the figure. Numeric tick labels are excluded from clipping and need three collisions to count, because the seed labels never mark them. Motivated by the 0/5 recall on the dense-text tasks. (b) Sort direction of a single bar series is checked in code (motivated by `regional-revenue-share`). (c) The legend bug from run 016 is fixed. (d) The facts JSON no longer truncates annotations, reports actual face colours, and the model is told not to do arithmetic and to leave ordering to the rules (motivated by run 082 and the Part 3 FPs).

## 4.2 Final Evaluation on Seed and New Tasks

See the two tables above. Seed macro-MCC rises from 0.332 to 0.500 and authored macro-MCC from −0.064 to 0.398. `hard_to_read` goes from −0.242 to 0.481 on seed (13 TP, 0 FP) and from −0.189 to 0.853 on authored. *Regression:* seed `wrong_data` falls from 0.157 to 0.041 (5 TP, 12 FP vs. 9 TP, 15 FP): forbidding arithmetic removed true positives as well as false ones.

## 4.3 Qualitative Analysis of Self-Authored Tasks

The change helped where the failure is measurable: all three `regional-revenue-share` runs are now caught by the sort rule, the 3B headcount chart by the squeeze check, and all three scatters by the overlap check. *False negative,* ***`lyon-rolling-temperature__glm`*** *(wrong_data):* the line's first values equal the daily values (8.9, 6.25), the signature of `min_periods=1`, but deciding that needs arithmetic the model is now told not to do; a rule for "no value for the first $N$ rows" (count leading NaNs) would catch it. *False negative,* ***`lyon__qwen`*** *(hard_to_read):* 86 horizontal date ticks overprint, yet nothing fired. Debugging the replayed manifest showed that in Matplotlib 3.10 tick labels report `.axes = None`, so my hook filed all 86 as free text; the free-text overlap check then skipped them because dates match my "numeric label" pattern. The rule was right and the role assignment was wrong. Fixed in Section 4.4. *False positive,* ***`cumulative-signups-by-channel__glm`*** *(wrong_chart):* the model marks "stack Organic, Paid, Referral" as violated while quoting the measurement `["Organic","Paid","Referral"]`, which satisfies it. Its verdict contradicts its own evidence, the same failure as Part 1's inverted YES, now inside the JSON.

## 4.4 Further Changes

**Tick-label roles.** Following the `lyon__qwen` miss, the hook now assigns roles from each axes' own tick-label lists and records each x tick's anchor pixel, so rotated labels are spaced by anchor rather than by their axis-aligned boxes. The readability judge makes no model call, and the other families

are unchanged by this fix, so the scores below replace only the `hard_to_read` decisions of the final run with the new deterministic ones (2 seed and 1 authored decisions changed):

| | Seed | | Authored | |
|---|---|---|---|---|
| Family | final | + roles fix | final | + roles fix |
| execution_failure | 1.000 | 1.000 | 0.000* | 0.000* |
| wrong_data | 0.041 | 0.041 | 0.739 | 0.739 |
| wrong_chart | 0.479 | 0.479 | 0.000* | 0.000* |
| hard_to_read | 0.481 | 0.523 | 0.853 | 1.000 |
| macro-MCC | 0.500 | 0.511 | 0.398 | 0.435 |

The seed gain (13→15 TP, still 0 FP) shows the fix is not specific to my tasks. Authored 1.000 is on 15 runs I labelled myself and should be read as "no counter-example found", not as solved.

Not done in this pass: a consistency filter that drops "violated" items whose quoted measurement matches the requirement; code-side checks for window and cumulative semantics; and an image-based readability fallback for 3D axes, whose projected text boxes are excluded from the measured checks.